

Proxima Compass Base - User Manual

1.9

Introduction	3
Product Overview	3
System requirements	3
Installation	4
Docker version	4
Prerequisites	4
Installation procedure	4
Industrial PC version	4
Connections	5
System Usage	5
Configuration	7
Connections	7
Kinematics	8
In-depth study on velocity and acceleration	9
Differential-drive	9
Omnidirectional	10
Path Following	10
Collision Avoidance	11
Obstacle Avoidance	11
Motion	12
OPC-UA	12
ROS2	14
Sensors	14
Saving the configuration	17
Download the configuration	17
Uploading the configuration	17
Testing the configuration	17
Odometric tests	18
Mapping	20
Running the mapping	20
Map Management and Editing	21
Commands	24
Missions	25
Commands	25
Robot localization	25
Mission Start	27
Possible Scenarios	29
Mission Control	30
Robot Status	31
Correct logout procedure	32
Troubleshooting	33
Warnings and precautions	34
Conclusion	34

Contacts	34
Appendix	34
Upload configuration	34

Introduction

This Manual describes the structure and operation of Proxima Compass Base, an operative system for mobile robots autonomous navigation. When properly configured, this system enables a robotic vehicle to localize itself using LIDAR sensors and execute point-to-point movement missions within its operational environment.

The manual includes detailed instructions for product installation and outlines the necessary vehicle specifications for compatibility. It also explains the intended use for optimal operation.

Product Overview

The Proxima Compass Base system consists of:

- a processing unit (industrial PC), to be connected to the robot's sensors/actuators and which can be interfaced via an Ethernet or Wi-Fi connection.
 - Alternatively, a Docker container can be installed inside the customer's processing unit. The latter must then be properly configured and connected to the robot's sensors/actuators and capable of interfacing via Ethernet or Wi-Fi.
- the on-board software in the Proxima Compass Base version which includes:
 - natural localization
 - environmental mapping using LIDAR sensors
 - point-to-point navigation

Proxima Compass Base is compatible with different types of robots, which must meet the following requirements.

Connection protocol to the motion part	LIDAR sensors with TCP/IP protocol	Kinematics
OPC-UA ROS2	SICK (nanoScan3) PEPPERL+FUCHS (R2000) DATALOGIC (Sentinel) PILZ (PSENscan) ROS2 based lidar	Differential Drive Omnidirectional

System requirements

Required disk space for installation	5 GB
Recommended RAM	4 GB
Recommended CPU	4 core, 3 GHz
Recommended disk space for logging	5 GB
Recommended operating system	Ubuntu Server 20.04 and later

Installation

Docker version

You need to install a single unit for each robot you want to manage via Proxima Compass Base.

Below is the installation procedure via script on the PC used to host the Docker version of Proxima Compass Base.

Prerequisites

1. the customer has created a specific user for the configuration and management of Proxima Compass Base (called "proxima")
2. The created user must have write permissions in the /etc/systemd/system folder to be able to add files with the .service extension that allow Proxima Compass Base to start correctly.
3. it is necessary that the operations below are performed by the user in question mentioned above (i.e., "proxima")
4. It is necessary that the date of the PC in charge has the date as per the calendar to avoid possible malfunctions

Installation procedure

- Download proxima_deploy.tar.gz to your PC;
- Extract the tar using the instruction
 - `tar -xzf proxima_deploy.tar.gz`
- After extracting it, the user will find in the directory where it is located a folder called "proxima_deploy". **The contents of this folder must not be modified in any way, even once the installation procedure is complete..**
- Run the install.sh script using the key provided for license management, via the statement
 - `./proxima_deploy/install.sh <key>` ;
- If everything went well, the user will see the message "Installation completed" on the screen. As a result, the script will create two new folders: "RawScan" and "compass_base_setup". **The contents of these folders must not be modified in any way, even once the installation procedure is complete..**
- Enable and launch the services needed to start the Proxima Compass Base application; If the user created to configure and manage Proxima Compass Base does not have the necessary permissions to execute the following commands, another user will need to execute them. You will need to enter the user password to execute them.
 - `sudo systemctl enable --now proxima-compass-base-db.service`
 - `sudo systemctl enable --now proxima-watchdog.service`

If you need to restart the application, you can run the following command:

- `sudo systemctl restart proxima-watchdog.service`

To check the application log you can run the following command:

- `sudo journalctl -u proxima-watchdog.service -f`

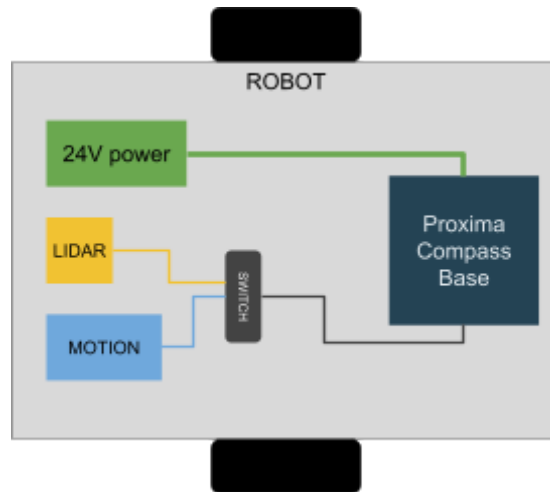
Industrial PC version

The Proxima Compass Base processing unit must be housed integrally with the robot you want to make autonomous. A single unit must be installed for each robot you want to control via Proxima Compass Base.

Connections

Proxima Compass Base must be connected:

- to a 24V power supply, using the connection found on the industrial PC.
- with an Ethernet cable to a switch that acts as a collector for the data from the LIDAR sensors and the motion part
 - see example connection below (the kinematics shown are for illustrative purposes only)



When you turn on your PC you will be able to use Proxima Compass Base.

System Usage

Once Proxima Compass Base is turned on and installed, type the IP address associated with the PC on which the system was installed, followed by port 5173, into the browser's search bar.

For example: 192.168.1.102:5173

The user will then be able to view the Login interface.

You can log in by entering your credentials and then pressing the enter key or clicking on the button **ENTER**.

Login credentials:

- **user:** admin
- **password:** admin

USE: If the error “No more authorized” appears cyclically, press **F5** to reset the cache.

Proxima Compass

a brain for your robot

Login

SIGN IN

Therefore, once you log in, you will be able to view the system homepage.

NOTE: If the login process is successful, a security token will be saved in the web browser's local storage; this token will be used for all subsequent requests to the system; this process is transparent to the user.



CONFIGURATION

MAPPING

MISSION

© 2025 Proxima Robotics S.r.l - v

When starting Proxima Compass Base for the first time, the user can only configure the system using the button *Configuration*.

The other options, *Mapping* and *Mission*, will be temporarily disabled until the initial setup is performed.

Configuration

By clicking on the button *Configuration* you can access the configuration interface.



General

Robot Name

agilex

Connections

IP

10.74.80.77

CIDR

24

Kinematics

Type

diff_drive

Wheel radius

0.08

m

Reduction

1

ratio

Axle length

0.49

m

Footprint a

0.306

m

Footprint b

0.306

m

Footprint c/d

0.29

m

Vel lin max

0.5

m/s

Vel ang max

0.5

rad/s

Acc lin max

0.3

m/s²

Acc ang max

0.35

rad/s²

Dec lin max

0.3

m/s²

Acc lat max

0.1

m/s²

Align acc ang max

0.32

rad/s²

Align vel ang max

0.4

rad/s

Align vel ang min

0.05

rad/s

Padding

0.05

m

Slowdown area vel max

0.3

m/s

Path Following

Out of Path Check

☒

Timeout

0

s

Check Distance

☒

Distance

0.2

m

Collision Avoidance

☒

Obstacle Stop Distance

0.5

m

Obstacle Avoidance

☒

Replan timeout

1

s

Motion

Type

ros2

Control Type

twist

Safety Topic

/safety

Odometry Status Topic

/odom

Odometry Status Frame

odom

Twist Cmd Topic

/cmd_vel

Sensors

sick_nanoscan3 - 10.74.80.7

ADD SENSOR

TEST

SAVE

DOWNLOAD TO FILE

UPLOAD FROM FILE

© 2025 Proxima Robotics S.r.l - v

Connections

The section *Connections* refers to the setting of the network interface that communicates with the sensors and motion part.

The value of this address must be set equal to the one on that specific network interface (corresponding to the one to which the Ethernet cable is connected). Classless Inter-Domain Routing (CIDR), on the other hand, depends on the bits that the user has dedicated to the routing prefix in their network (usually the default value, and therefore the one to be entered, is 24).

Example scenario: let's assume that the IP address of the PLC (or equivalent instrumentation) is 192.168.1.2; let's assume that the IP address of the LIDAR (or equivalent instrumentation) is 192.168.1.3 and the IP address of the Industrial PC where Proxima Compass Base resides is 192.168.1.10; the address that must be entered in the Connections field will be 192.168.1.10.

Please note: in the case of sensors configured with the IP address of the machine where the driver software resides, the IP address reported in the Connection field will be used for this purpose (e.g. SICK and Datalogic sensors).

Connections

IP

192.168.1.10

CIDR

24

Kinematics

The section *Kinematics* refers to the setting of the quantities necessary for the correct calculation of the robot's movements based on the kinematic model selected from the menu *type*. Some of these variables are shared between the various kinematics. These variables are listed in the table below.

Variable	Typology	Meaning
Wheel Radius	to be measured	Radius of the drive wheels (in meters)
Reduction	to be measured	Reduction ratio between wheel revolutions and engine revolutions.
Axle Length	to be measured	Length of the axle connecting the drive wheels (in meters)
Footprint a	to be measured	Robot footprint measured from axle center, front section (in meters), see Differential-drive section for a graphical explanation.
Footprint b	to be measured	Robot footprint measured from axle center, rear section (in meters), see Differential-drive section for a graphical explanation.
Footprint c/d	to be measured	Robot footprint measured from axle center, left and right sections (in meters), see Differential-drive section for a graphical explanation.
Vel lin max	parameter	Maximum allowable forward speed (in meters per second)
Vel ang max	parameter	Maximum allowed angular steering speed (in radians per second)
Acc lin max	parameter	Maximum allowable forward acceleration (in meters per square second)
Acc ang max	parameter	Maximum allowable steering acceleration (in radians per second frame)
Dec lin max	parameter	Maximum permitted deceleration (in meters per second squared)
Acc lat max	parameter	Maximum allowable lateral acceleration (in meters per square second). The lower the value, the more the robot slows down while steering.
Align acc ang max	parameter	Maximum angular acceleration to be maintained during a pure rotational movement (alignment)
Align vel ang max	parameter	Maximum angular velocity allowed during a pure rotational movement (alignment)
Align vel ang min	parameter	Minimum angular velocity allowed during a pure rotational movement (alignment)
Padding	parameter	It is the distance (in meters) that the robot must consider on all sides of its footprint as additional bulk.
Slowdown area vel max	parameter	It is the maximum speed (in meters per second) to be maintained within a slowdown area

ATTENTION: To select the maximum speed, take into account the working frequencies of the sensors and the motion part. In the absence of such information, it is recommended not to use a maximum speed higher than 0.5 m/s.

NOTE: as a general rule, we can assume that we don't want the robot to move more than 5 cm without receiving a position update from the localization system. The localization system updates with the lesser frequency between sensor data (lidar) and motion data (encoder). Therefore, given X the minimum update frequency between sensors and motion, the maximum allowed speed must be equal to X times 0.05 m.

Minimum frequency between sensors and motion	Maximum speed allowed
5 Hz	<u>The system cannot perform adequately</u>
10 Hz	0.5 m/s
25 Hz	1.25 m/s
30 Hz	1.5 m/s
50 Hz	2.5 m/s

ATTENTION: Velocities and accelerations must be all positive.

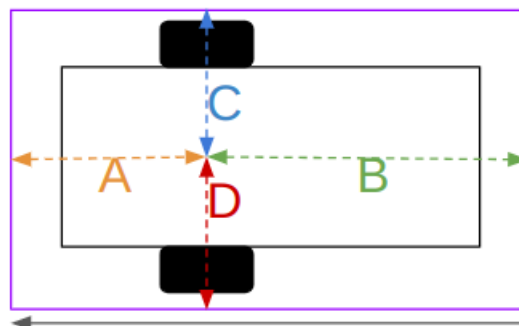
In-depth study on velocity and acceleration

During a mission, based on the state of the robot in terms of position and velocity (linear and angular), the control actions to be implemented are calculated such that the following requirements are satisfied:

- the linear velocity cannot be greater than `vel_lin_max` and must provide a linear acceleration (or deceleration) that is not greater than `acc_lin_max` (or `dec_lin_max`);
- the angular velocity cannot be greater than `vel_ang_max` and must include an angular acceleration (or deceleration) that is not greater than `acc_ang_max`;
- the lateral acceleration cannot be greater than the one set (`acc_lat_max`); if this condition is violated, the linear velocity is scaled in order to satisfy this constraint
 - It is suggested to act on the configurable parameter `acc_lat_max` to modify the robot's behavior when cornering rather than `vel_lin_max` or `vel_ang_max`.

Differential-drive

At the moment there are no specific variables for differential kinematics other than the shared ones. Below is a graphical example for a better understanding of `footprint_a`, `footprint_b`, `footprint_c_d`, the robot front is on the left.



For completeness, an example of how the form relating to the kinematics of a differential drive robot could be filled out:

Kinematics

Type

diff_drive

Wheel radius

0.08

m

Reduction

1

ratio

Axle length

0.49

m

Footprint a

0.306

m

Footprint b

0.306

m

Footprint c/d

0.29

m

Vel lin max

0.5

m/s

Vel ang max

0.5

rad/s

Acc lin max

0.3

m/s²

Acc ang max

0.35

rad/s²

Dec lin max

0.3

m/s²

Acc lat max

0.1

m/s²

Align acc ang max

0.32

rad/s²

Align vel ang max

0.4

rad/s

Align vel ang min

0.05

rad/s

Padding

0.05

m

Slowdown area vel max

0.3

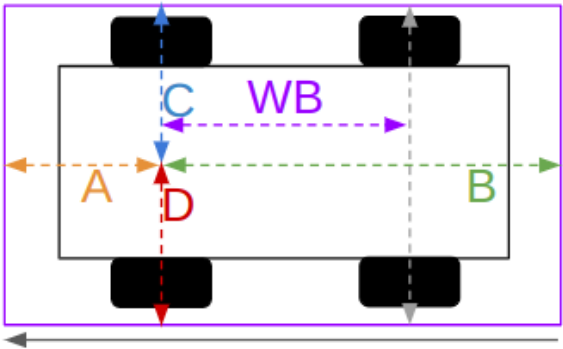
m/s

Omnidirectional

In the case of an omnidirectional vehicle, the variables to consider will be all of the above plus one:

Variable	Meaning
Wheel base	Distance between front and rear axle (in meters)

A graphical example is shown below for better understanding of footprint_a, footprint_b, footprint_c_d and wheel_base, the robot front is on the left.



For completeness, an example of how the form relating to the kinematics of an omnidirectional robot could be filled out:

Kinematics

Type

omnidirectional

Wheel radius

0.0762

m

Reduction

10

ratio

Axle length

0.545

m

Footprint a

0.28

m

Footprint b

0.28

m

Footprint c/d

0.29

m

Vel lin max

0.5

m/s

Vel ang max

0.5

rad/s

Acc lin max

0.2

m/s²

Acc ang max

0.1

rad/s²

Dec lin max

0.1

m/s²

Acc lat max

0.1

m/s²

Align acc ang max

0.1

rad/s²

Align vel ang max

0.1

rad/s

Align vel ang min

0.01

rad/s

Padding

0.2

m

Slowdown area vel max

0.2

m/s

Wheelbase

0.545

Path Following

Path Following

Out of Path Check

☒

Timeout

0

s

Check Distance

☒

Distance

0.2

m

In this section the user can change the robot's behavior while following the trajectory, currently it is possible to:

- enable or disable the trajectory tracking control using the option **Out of Path Check**, in this way the robot can check whether, for example, it has gone off the path due to a slip
- set the **Timeout** after which the robot recalculates the path after going off course
- enable or disable the control on the distance from the path using the option **Check Distance**.
 - **NOTE: if Out of Path Check is enabled and Check Distance is not enabled, the robot will not perform the off-path check**
- set the maximum distance from the path beyond which the robot is considered off-path using the variable **Distance**

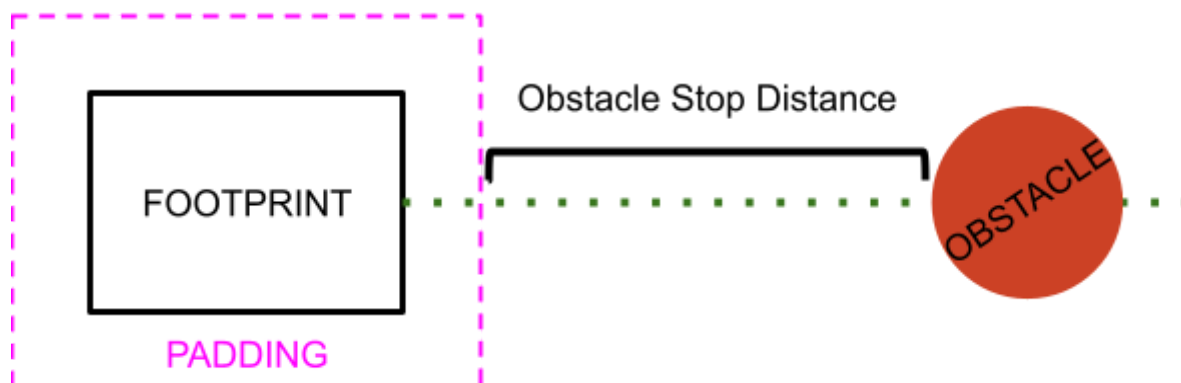
Collision Avoidance

Collision Avoidance ☒

Obstacle Stop Distance m

In this section the user can change the robot's behavior in terms of avoiding collisions with obstacles it encounters on its path. Currently, it is possible to:

- enable or disable collision checking along the trajectory using the option **Collision Avoidance**, in this way the robot can check if there could be a collision due to an unexpected obstacle along its path and stop
- set the distance at which the robot must stop from an obstacle on its path using the parameter **Obstacle Stop Distance**, the robot will stop at the set distance considering both the footprint and the padding set.



Obstacle Avoidance

Obstacle Avoidance ☒

Replan timeout s

In this section the user can change the robot's behavior with respect to avoiding obstacles it finds on its path. Currently it is possible to:

- enable or disable the avoidance of obstacles along the trajectory using the option **Obstacle Avoidance**, in this way the robot, once stopped due to collision avoidance, can calculate an alternative path
- set the time after which the robot plans the alternative route using the option **Replan Timeout**

Motion

The section *Motion* refers to the communication protocol setting between Proxima Compass Base and the component (hardware or software) which handles the control and measurement of the connected motors and encoders. The communication protocol, where applicable, is selected via a dedicated menu *type*. The combination of kinematics type, communication protocol, and control type will allow you to define specific control variables for the movement. Currently, not all combinations of kinematics, protocol type, and control are possible. An example is provided below.

OPC-UA

If you choose OPC-UA mode, you must set the IP address where the PLC resides, the port of the OPC-UA server active on the PLC itself (or equivalent instrumentation) and the variable *safety* which reports the value of a bit that signals a safety stop for the robot, measured by the motion part; usually the default active port for the OPC-UA protocol is 4840.

ATTENTION: *safety*, in the OPC-UA server, must be of type *byte*.

Next you have to define how the type of control takes place, whether by direct control of the motors (option *motors*) or via a velocity vector (option *twist*), with the variable *control_type*.

In the case of differential kinematics with communication protocol *opcua* and type of control *motors*, you will have to set the names of the OPC-UA variables for reading/writing wheel RPM data to and from Proxima Compass Base. The variables to be configured are shown in the table below.

Variable	Typology	Optional	Meaning
RPM Cmd Left	write	no	the desired speed in RPM for the left wheel, calculated by Proxima Compass Base
RPM Cmd Right	write	no	the desired speed in RPM for the right wheel, calculated by Proxima Compass Base
RPM Status Left	read	no	the speed measured in RPM at the left wheel, measured from the motion part
RPM Status Right	read	no	the speed measured in RPM at the right wheel, measured from the motion part

It is important that in defining each of the 5 OPC-UA variables (including *safety*) just presented, the entire path (defined by the names of the OPC-UA nodes traversed) that defines where the individual OPC-UA variable is located within the OPC-UA server in the PLC is reported. This path is defined by the names of the OPC-UA nodes traversed in the OPC-UA server object tree until reaching the OPC-UA variable in question, separated from each other by the "/" character.

ATTENTION: such variables, in the OPC-UA server, must be of type *float*.

Other parameters to configure are:

Variable	Typology	Optional	Meaning
Safety	read	no	safety state active or inactive
Alive	write	yes	a variable set to true anytime a read/write operation is performed by Proxima Compass Base
Front Left Distance	write	yes	distance from the nearest obstacle in the upper left quadrant with respect to the direction of motion of the robot

Front Right Distance	write	yes	distance from the nearest obstacle in the upper right quadrant with respect to the direction of motion of the robot
Rear Left Distance	write	yes	distance from the nearest obstacle in the lower left quadrant relative to the direction of motion of the robot
Rear Right Distance	write	yes	distance from the nearest obstacle in the lower right quadrant with respect to the direction of motion of the robot

As an example, the following image shows how the section *Motion* looks like in the case of *opcua* protocol and *motors* control type:

Motion

Type

opcua

Control Type

motors

OPC-UA Server IP

127.0.0.1

OPC-UA Server Port

4840

Alive (optional)

Objects/PLC1/alive

Safety

Objects/PLC1/safety

Front Left Distance (optional)

Objects/PLC1/FL_dist

Front Right Distance (optional)

Objects/PLC1/FR_dist

Rear Left Distance (optional)

Objects/PLC1/RL_dist

Rear Right Distance (optional)

Objects/PLC1/RR_dist

RPM Cmd Left

Objects/PLC1/cmdL

RPM Cmd Right

Objects/PLC1/cmdR

RPM Status Left

Objects/PLC1/statusL

RPM Status Right

Objects/PLC1/statusR

In the case of **omnidirectional kinematics** with communication protocol ***opcua***, and type of control type ***motors***, you will need to set the names of the OPC-UA variables for reading/writing wheel RPM data to and from Proxima Compass Base.
The variables to be configured are represented in the table below.

Variable	Typology	Optional	Meaning
RPM Cmd Front Left	write	no	the desired speed in RPM for the left front wheel, calculated by Proxima Compass Base
RPM Cmd Front Right	write	no	the desired speed in RPM for the right front wheel, calculated by Proxima Compass Base
RPM Cmd Rear Left	write	no	the desired speed in RPM for the left rear wheel, calculated by Proxima Compass Base
RPM Cmd Rear Right	write	no	the desired speed in RPM for the right rear wheel, calculated by Proxima Compass Base
RPM Status Front Left	read	no	the speed measured in RPM at the left front wheel, measured from the motion part
RPM Status Front Right	read	no	the speed measured in RPM at the right front wheel, measured from the motion part
RPM Status Rear Left	read	no	the speed measured in RPM at the left rear wheel, measured from the motion part
RPM Status Rear Right	read	no	the speed measured in RPM at the right rear wheel, measured from the motion part

It is important that in the definition of each of the 9 OPC-UA variables (including *safety*) just presented, the entire path (defined by the names of the OPC-UA nodes traversed) that defines where the individual OPC-UA variable is located within the OPC-UA server in the PLC is reported. This path is defined by the names of the

OPC-UA nodes traversed in the OPC-UA server object tree until reaching the OPC-UA variable in question, separated from each other by the “/” character.

ATTENTION: such variables, in the OPC-UA server, must be of type *float*.

ROS2

In the case of **differential kinematics** with communication protocol *ros2*, and type of control type *twist*, the following variables will need to be set.

Variable	Typology	Meaning	Type
Twist Cmd Topic	publisher	topic for speed command	geometry_msgs/msg/Twist
Odometry Status Topic	subscriber	topic for the state of odometry	nav_msgs/msg/Odometry
Odometry Status Frame	parameter	frame for odometry status	string
Safety Topic	subscriber	topic for sending safety status	boolean

As an example, the following image shows how the section *Motion* looks like in the case described:

MOTION

type

ros2

control_type

twist

safety_topic *

/safety

twist_cmd_topic *

/cmd_vel

odometry_status_topic *

/odom

odometry_status_frame *

odom

Sensors

By pressing the “ADD SENSOR” button in the section *Sensors* you can add one or more LIDAR sensors for navigation. Remember, at least one sensor must be added.

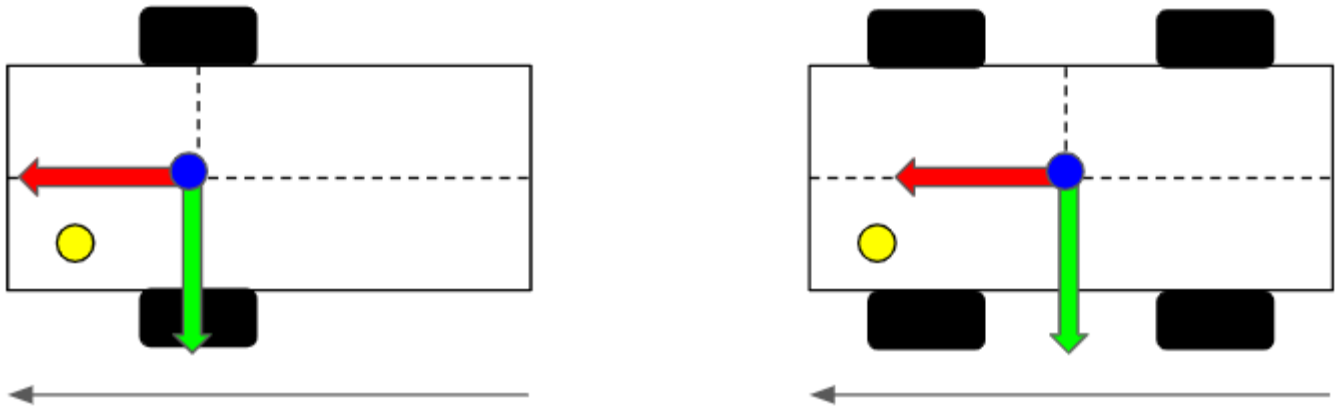
Sensors

ADD SENSOR

For each added sensor you will need to specify:

Variable	Meaning
type	Sensor type; the following models are currently integrated: sick_nanoscan3 (model nanoScan3, brand SICK), datalogic (model Sentinel, brand Datalogic), pf_r2000 (model R2000, brand Pepperl+Fuchs), ros2_scanner(if you already have a laser that is integrated with ros2), pilz_psenscan (model PSENscan, brand PILZ)
x	sensor placement relative to the axle center, longitudinal axis (in meters)
y	sensor placement relative to axle center, lateral axis (in meters)
z	sensor placement relative to the axle center, vertical axis (in meters)
yaw	sensor orientation with respect to the longitudinal axis of the robot (in degrees), positive in a counterclockwise direction

Please note that the sensor placement variables (variables x , y , z (of the table above) must be considered with respect to the center of the robot's reference system (which will generally be different from model to model). In the case described below, the reference system is placed at the center of the axle for differential kinematics and at the center of the axes (both in the longitudinal and lateral directions) for omnidirectional kinematics. An example image is shown below for differential and omnidirectional kinematics (the yellow dot represents a hypothetical positioning of the LIDAR sensor).



The image above represents the positioning of the reference system that is usually considered for differential and omnidirectional kinematics respectively. Remember that the direction with positive x is the longitudinal north direction (red arrow), the direction with positive y is the lateral west direction (green arrow), the direction with positive z is the vertical upward direction (blue arrow, since this is a top view, a circle is displayed).

Depending on the type of laser chosen via `type`, you will then have to specify further variables.

Variable	Meaning
<code>ip</code>	Sensor IP address
<code>min_ang</code>	minimum angle with respect to the LIDAR sensor's data readout range (in degrees). Setting an angle of 0 degrees means that the sensor's measurements will be considered starting from the frontal direction of the sensor itself up to the Max Ang measurement.
<code>max_ang</code>	maximum angle with respect to the LIDAR sensor's data readout range (in degrees). Setting an angle of 0 degrees means that the sensor's measurements will be considered from the minimum to the frontal direction of the sensor itself.
<code>topic</code>	This is the ros2 topic where you can read the scan to use while browsing.
<code>frame</code>	is the frame used to generate the reference system for the laser
<code>is_master</code>	Boolean value that identifies this sensor as master if checked
<code>index</code>	integer value that identifies the index in the sensor network, 0 is the master

Complete table:

	<code>datalogic_sentinel</code>	<code>sick_nanoscan3</code>	<code>pf_r2000</code>	<code>ros2_scanner</code>	<code>pilz_psenscan</code>
<code>type</code>	✓	✓	✓	✓	✓
<code>id</code>	✓	✓	✓	✓	✓

x	✓	✓	✓	✓	✓
and	✓	✓	✓	✓	✓
With	✓	✓	✓	✓	✓
yaw	✓	✓	✓	✓	✓
ip	✓	✓	✓		✓
min_ang	✓	✓			✓
max_ang	✓	✓			✓
topic				✓	
frame				✓	
is_master					✓
index					✓

As an example, the following image illustrates an example where you want to add a Datalogic type sensor with IP address 127.0.0.1.

New sensor

Type
datalogic_sentinel

X
0

Y
0

Z
0

YAW
0

IP
127.0.0.1

Min angle
0

Max angle
0

CANCEL

SUBMIT

Next to each of the added sensors the user will notice two icons, representing (from left to right) the possibility of modifying or deleting a sensor previously added to the configuration.
If the edit button is selected the user will be able to correct the previously entered variables..

ros2_scanner - no ip



datalogic_sentinel - 127.0.0.1



If the button represented by the trash can icon is selected, the user will delete the sensor from the list, causing it to no longer appear in the system configuration.

Saving the configuration

With a click of a button “SAVE” At the bottom the configuration is sent to the robot and will become the configuration used from that point on.



If the save is unsuccessful, the user will see a warning specifying the type of error that occurred.

Download the configuration

The user can decide to download the configuration present in Proxima Compass Base to his PC by clicking on the “DOWNLOAD TO FILE” button from the red arrow in the image below (the file will have the extension *json*).



Uploading the configuration

If the user has a valid configuration for Proxima Compass Base on their PC, then it is possible to upload this configuration, so that it can be saved and the robot can be made operational with the loaded parameters. The button “UPLOAD FROM FILE” allows you to navigate through the directories of your PC and select the preferred configuration (the file must have the extension *json*).

ATTENTION: you will have to use the syntax as reported in the Appendix, section Upload configuration

Testing the configuration

With a push of a button *Test* it is possible to enter the test page of the last saved configuration, so that you can check that the system is functioning correctly and receive any instructions for correcting the configuration.



The test page presents a schema where you can:

- Verify that the PC, PLC (or alternative system), and sensors related to the last saved configuration are currently active by performing a ping operation. This way, the user can be sure that all components are communicating with each other correctly.
- Verify that the added OPC-UA variables (if you have chosen the OPC-UA communication protocol) have been found and registered to ensure the correct functioning of the part of *Motion* of the system.
 - If a different communication protocol has been chosen, the result of this test is undefined.
- Perform a linear odometry test: by providing a distance (in meters) to the system in the available text box, the user can verify that the distance traveled by the robot matches the one manually set and guarantee the accuracy of the linear odometry measurement.
- Perform a rotation odometry test: similarly to the previous case, the user can provide a distance value (in degrees or radians) within the text window provided and verify the accuracy of the odometric

measurement by comparing the value returned on the screen by Proxima Compass Base and the value traveled in the field.

- Verify that data from the laser sensors is reaching the Proxima Compass Base using the “Scan check” routine.

Below is a representation of the test table that the user can observe, on the specific dedicated page, upon first startup.

Distance		Automatic Test Routine	Message	Result
		Ping PC	Ok	
		Ping Sensors	Ok	
		Scan check	Ok	
	 Enter distance here ☐ Radians?	[°] Odometry Rotation Check	Ok	43.99
	 Enter distance here	[m] Odometry Translation Check	Ok	0.59

© 2025 Proxima Robotics S.r.l - v

Once completed, the test result will be visible in the results column, while the *Message* will display a text message confirming success or any errors encountered during the test.

ATTENTION: the number and which tests are displayed depends on the chosen configuration; see the API manual for details

Odometric tests

Once the distance (linear or angular) has been set and the button pressed *Play* of the test, the user will see the robot move. In the column *Result* the distance covered by the robot up to that point will appear (calculated via odometry).

Once the robot has travelled the indicated distance it will stop and the number in the column *Result* will not change again until the next odometric test, so as to give the operator time to measure the distance actually travelled by the robot and verify that it is approximately equal to that indicated in the column *Result*. From the moment an odometry test is started the user can decide to interrupt it (thus stopping the robot's movement) at any time by pressing the button *Stop* which is located in the column next to the button *Play*.

ATTENTION: For linear and angular odometry tests, it is the user's responsibility to enter the correct distance values based on their needs and constraints of where the robot resides.

ATTENTION: it is strongly recommended to keep the safety system active so that it can intervene if the odometry test ends up too close to obstacles

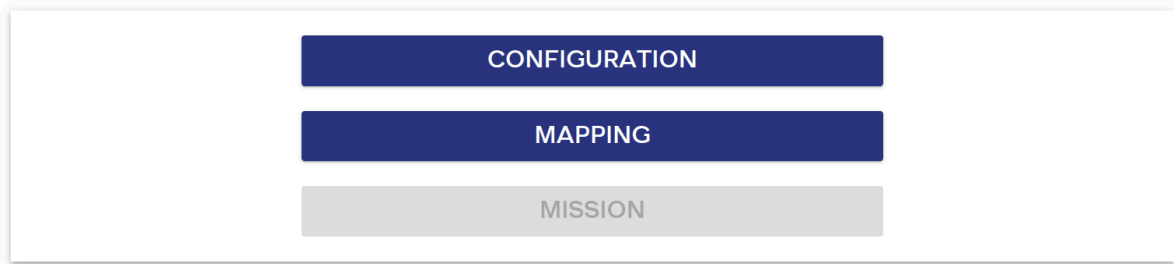
The button at the top left with the arrow drawn inside allows you to return to the configuration page.

✔ Configuration loaded correctly.

Distance		Automatic Test Routine	Message	Result
		Ping PC	Ok	
		Ping Sensors	Ok	
		Scan check	Ok	
	 Enter distance here ☐ Radians?	[°] Odometry Rotation Check	Ok	43.99
	 Enter distance here	[m] Odometry Translation Check	Ok	0.1

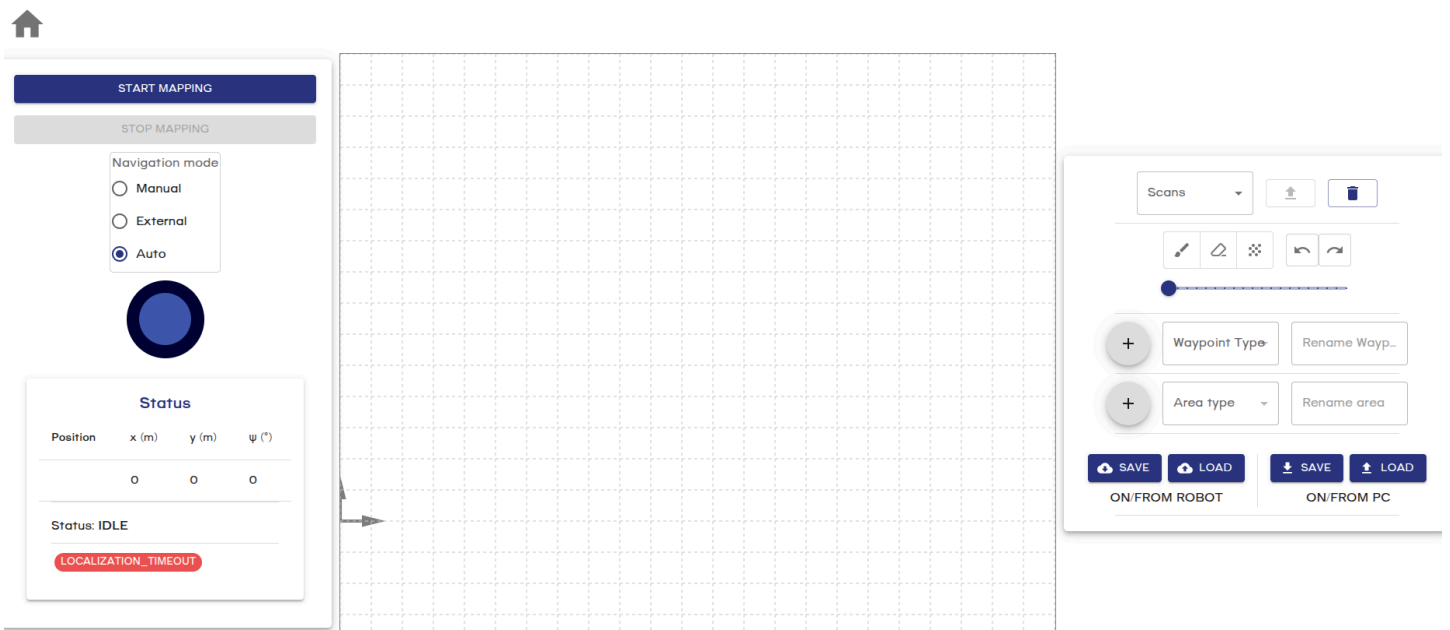
Mapping

Once the system configuration has been successfully saved, the user can proceed to the mapping phase. By pressing the button *Mapping* you access the mapping interface.



© 2025 Proxima Robotics S.r.l - v

Once opened, the environment mapping page looks like the one shown in the following image:



© 2025 Proxima Robotics S.r.l - v

NOTE: The system will report a `LOCALIZATION_TIMEOUT` error, which is normal since the mapping is not running

Running the mapping

The steps to perform a mapping are:

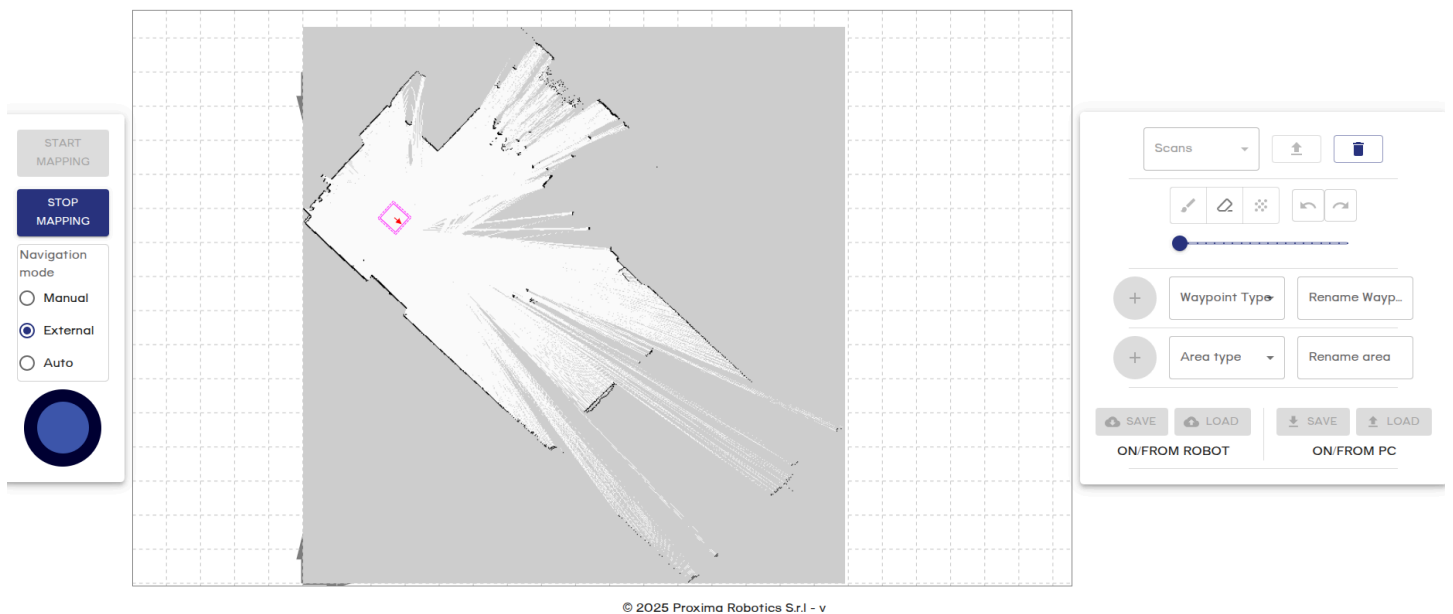
- press the button *START* to start the mapping process.
- To move the robot in the environment and therefore allow mapping, you can use your own appropriately configured physical joystick or the virtual joystick made available by the graphical interface
 - **ATTENTION:** Movement via a physical joystick is not managed by Proxima Compass Base. Therefore, it is the customer's responsibility to develop an architecture such that moving the joystick directly activates the wheels (without going through Proxima Compass Base).
- To activate the joystick, click "Manual" just above it, which switches to manual navigation. From this point until automatic mode is restored, the robot can be moved via the graphical interface.

Navigation mode

- ☒ Manual
- ☐ External
- ☐ Auto

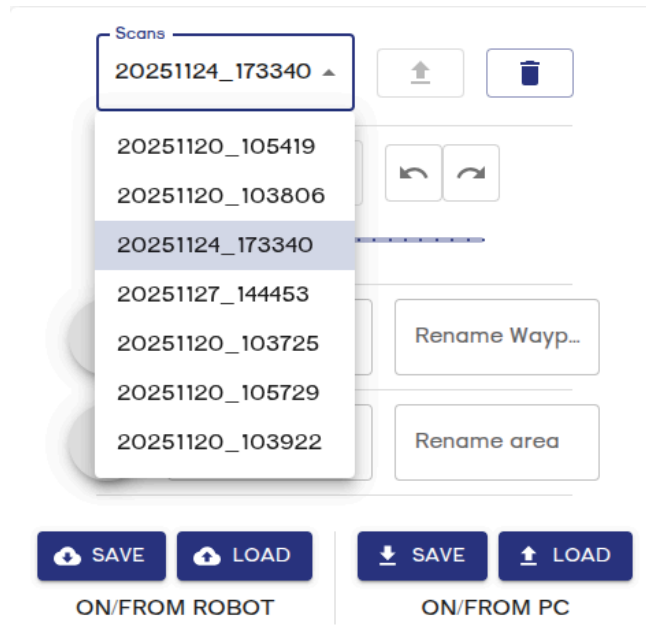
- approximately once every 5 seconds an updated version of the map that Proxima Compass Base is creating will appear
- When you think you've finished, press the button *Stop* to stop the mapping process and return the robot to automatic mode by pressing “Auto”.
- By pressing “External”, it is possible to use an external joystick (appropriately configured) to move the robot.

Example of mapping procedure:

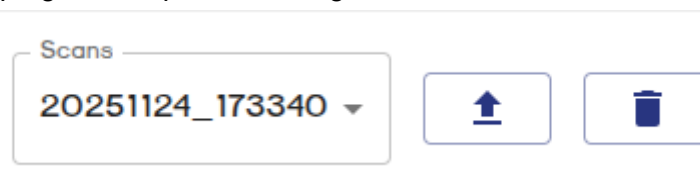


Map Management and Editing

Once the mapping is stopped, the generated image will appear in the list of available scans.



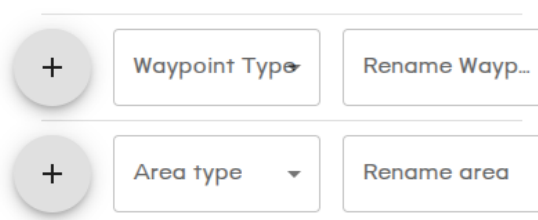
The map image, whose name will be composed of the date and time it was generated, can be selected from the drop-down menu at the top right and uploaded using the button with the arrow at the top.



The user can also delete one of the scans generated by the mapping process by clicking on the dedicated trash can button shown below after selecting the scan to be deleted from the drop-down menu.

At this point the scan will no longer be present within the system.

Once you have loaded the map you want to use, it is possible to add and modify waypoints and areas using the section summarised below and which will be explained in detail later.



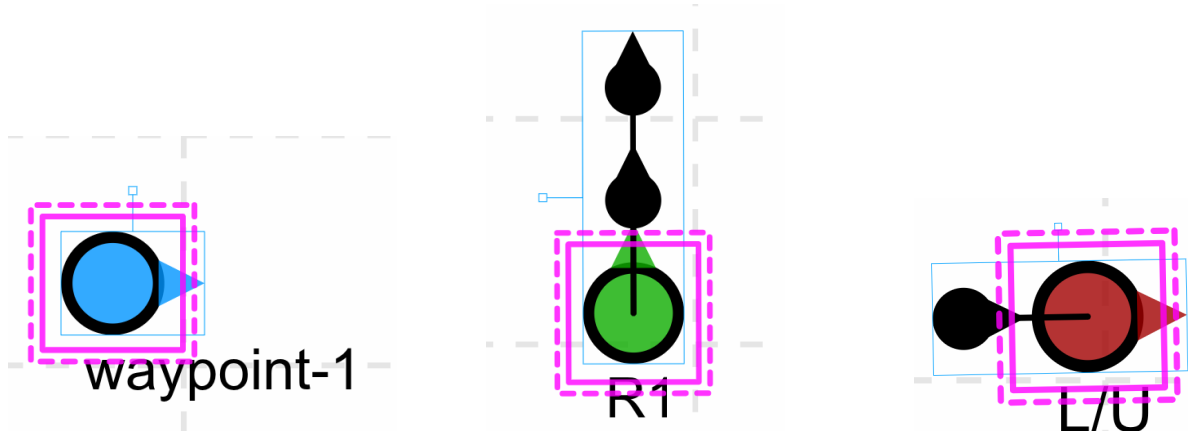
Below are the possible operations on points of interest and areas.



Adds a point of interest (waypoint) or an area:

- By clicking on the “+” button, a new waypoint or rectangular area will appear in the image
- in the case of a polygonal area the user will have to click for each vertex
- drag the waypoint or area to the desired location with the left mouse button

- the waypoint or area can be rotated with the appropriate tool

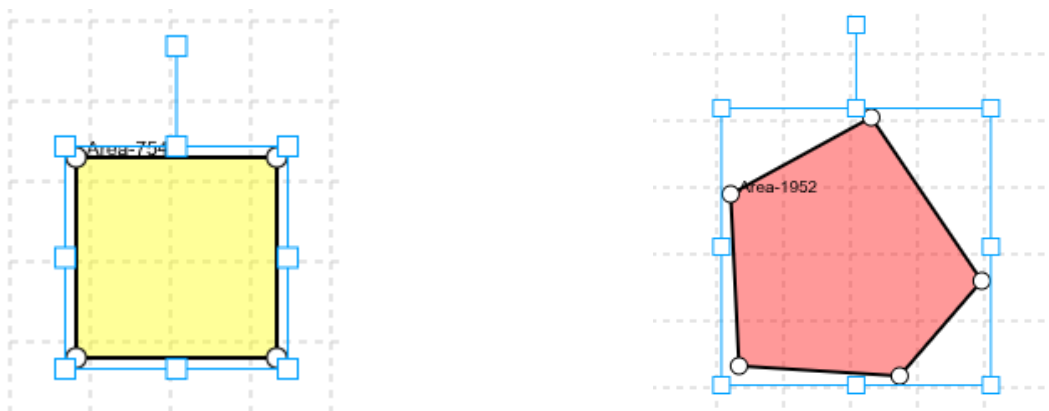


The currently selectable waypoint types are:

- **target**: the robot travels to this waypoint (blue) with the same precision as the localization precision
- Docking Stations
 - **recharge_station**: the robot travels to the first predock point and then execute the docking to reach the final destination (green)
 - **load_unload_station**: the robot travels to the first predock point and then execute the docking to reach the final destination (red)

The maneuver can be configured to happen both forward or backward in the dedicated menu, the control can be based both on odometry and visual add (the final precision does not depend on the localization precision) depending on the docking type

- **simple**: a docking maneuver based on odometry motions, final precision depend on the precision on which the robot arrived on the first predock point
- **reflector**: (to be implemented) a docking maneuver based on odometry motions in feedback with visual aid given by reflectors tracked by the lidar sensor
- **geometry**: (to be implemented) a docking maneuver based on odometry motions in feedback with visual aid given by a geometric shape tracked by the lidar sensor



Currently the selectable area types are:

- **slowdown_areas**: the robot slows down within this type of area
- **interdicted_areas**: the robot will never enter this type of area

After selecting a waypoint from the drop-down menu, you can rename it by entering the new name in the “Rename” text field, the user will have the different options depending on the selected object type.

Targets

Waypoint Type

target

Rename Waypoint

waypoint-1

Recharge and Load / Unload stations

Waypoint Type

load_unloa...

Rename Waypoint

L/U

Forward Docking

Predock Distances (m)

0.5

Docking type

simple

reflectors

geometry

Areas

Area type

slowdown_...

Rename area

area-1

The user can also, after selecting it using the appropriate drop-down menu, delete a waypoint from the map using the delete key on the keyboard.



- The menu contains the following drawing tools:
 - **BLACK PEN (black)**:freehand drawing in black (for example to represent a space occupied by obstacles)

- **EMPTY SPACE (gray)**: freehand drawing in gray (for example to represent an unfamiliar environment)
- **ERASER (white)**: freehand drawing in white (for example to represent a space free from obstacles)
- **PEN STROKE WIDTH**: slider that allows you to change the thickness of the pen and eraser

Additionally there are UNDO and REDO buttons.



- **SAVE ON ROBOT**: save mapping and navigation data to the robot
- **LOAD FROM ROBOT**: load the currently used mapping and navigation data from the robot
- **SAVE ON PC**: save mapping and navigation data to the user's PC
- **LOAD FROM PC**: load mapping and navigation data from the user's PC

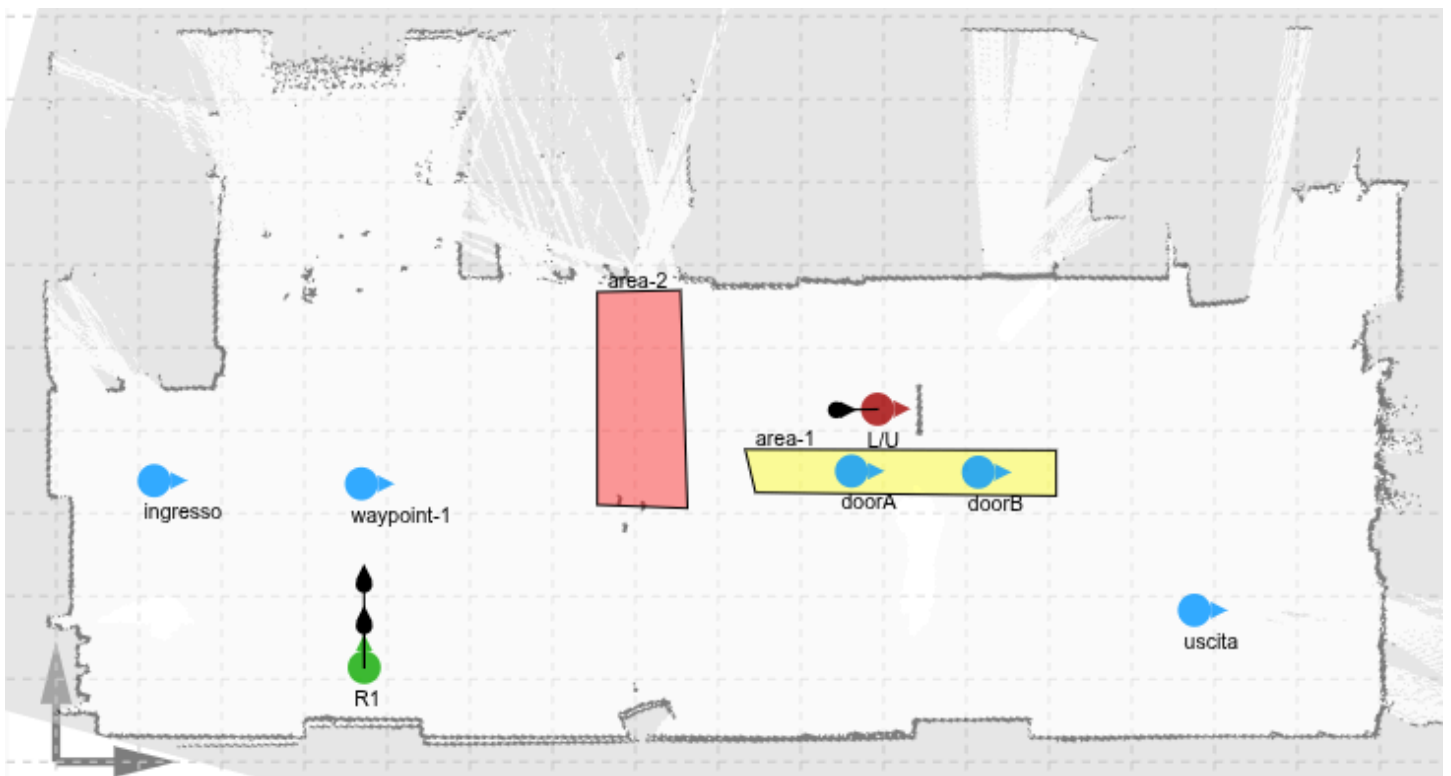
USE: In the current version of the software it is important to first position the map as desired, and only then make the drawings by hand, as these are not integral with the map itself.

Commands

- **Zoom**: wheel del mouse
- **Mouse pan**: right click del mouse

ATTENTION: Saving mapping data deletes previously saved data.

Here is an example of an edited map.



Missions

Once the navigation map has been saved successfully, the user can proceed to the mission phase.



CONFIGURATION

MAPPING

MISSION

© 2025 Proxima Robotics S.r.l - v

By pressing the button *Mission* you access the mission interface. The interface offers the possibility to:

- Locate the robot on the navigation map manually or on a waypoint.
- Assign a navigation mission to one of the points of interest (waypoints) added during the map construction phase.

Navigation mode

Manual

External

Auto

Status

Position	x (m)	y (m)	ψ (°)
	13.76	1.79	0.17

Velocity	x (m/s)	y (m/s)	ψ (rad/s)
Req	0	0	0
Actual	0	0	0

Status: IDLE

No errors

Properties

Show Obstacles

Localization

Manual

uscita

Mission Queue

Select waypoint

ADD

Commands

- **Zoom:** wheel del mouse
- **Mouse pan:** right mouse button

Robot localization

Once you enter the page *Mission* the robot is in the Waiting Navigation State (i.e., *IDLE*), the localization error will also be present at the same time *LOCALIZATION_TIMEOUT*.

Mission status

Position	x (m)	y (m)	ψ (°)
	0	0	0

Velocity	x (m/s)	y (m/s)	$\dot{\psi}$ (rad/s)
Req	0	0	0
Actual	0	0	0

Status: IDLE

LOCALIZATION_TIMEOUT

Localization can be done in 2 ways: manual localization or by waypoint.

Manual localization is done by pressing the button *MANUAL* in the section *LOCALIZE*. At this point you can set the current position and orientation of the robot.

The user must therefore:

- click on the button *MANUAL* and then move the cursor into the image.



- Hold down the left mouse button at the point within the image where the robot is located and drag it, while still remaining within the image: an arrow will appear with the tail indicating the position on the map where the robot is located and the head indicating the direction in which the robot is looking.
- Release the mouse once you find the desired orientation.

If the localization procedure is successful, the measurements detected by the laser sensors (represented by red pixels) will appear on the screen and the localization error will disappear (*LOCALIZATION_TIMEOUT*) previously reported by the system.

If the user is not satisfied with the location, he can repeat this operation in order to overlap the laser measurements as much as possible with the boundaries outlined on the map.

Waypoint-based localization can be done by selecting a waypoint from the drop-down menu always located in the section *LOCALIZATION* and pressing the button *AT WPS*. If the procedure is successful, the result will be that the robot will locate with the same position and orientation as the selected waypoint.



ATTENTIONIt is strongly discouraged to relocate the robot during the execution of a point-to-point transfer mission. The correct procedure is to abort the mission (see below for details), relocate the robot following the procedure described above, and reassign a navigation mission.

Mission Start

To start the mission the user must insert at least one task into the queue "*MISSION QUEUE*". To do this, the user selects the desired waypoint from those available, the interpolation radius (which must be greater than or equal to 0) in the appropriate field (in meters) and presses "*ADD*". The waypoints coincide with those added during the map editing phase after the mapping phase.

Mission Queue

WP-6 ▾

Radius
0.2

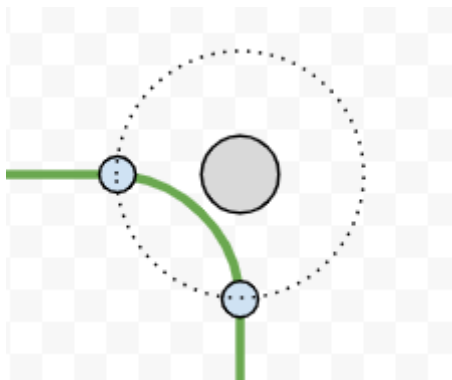
ADD

⋮

WP-6
Radius: 0.2

🗑

RADIUS: the interpolation radius indicates the radius of the circle constructed around the point of interest; this circle intersects the broken lines of the path incident on the point of interest; the intersection points are those used to construct the curvilinear line joining the two broken lines and thus obtain an interpolated line; for the first and last points of the sequence, this value does not affect the behavior.



NOTE: if you use a radius equal to 0 the robot will reach the selected waypoint in position and orientation before proceeding to the next waypoint without performing any interpolation.

ATTENTION: avoid placing points opposite each other and setting precision other than zero. this could cause the robot to attempt an excessively sharp turn, making sudden maneuvers.

Below the ADD button is a list of the tasks the robot will perform.

Mission Queue

WP-7 ▼

Radius

0.2

ADD

WP-6

Radius: 0.2

WP-2

Radius: 0.2

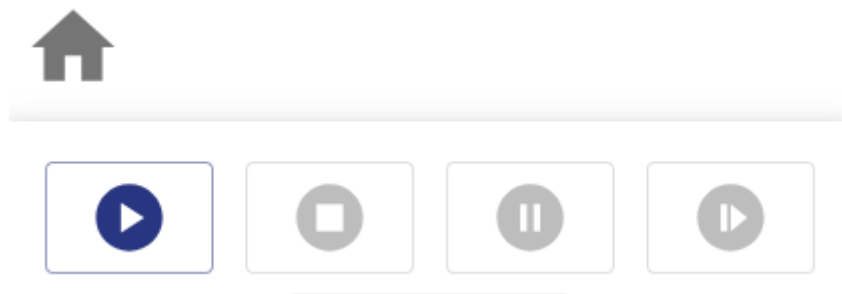
WP-3

Radius: 0.2

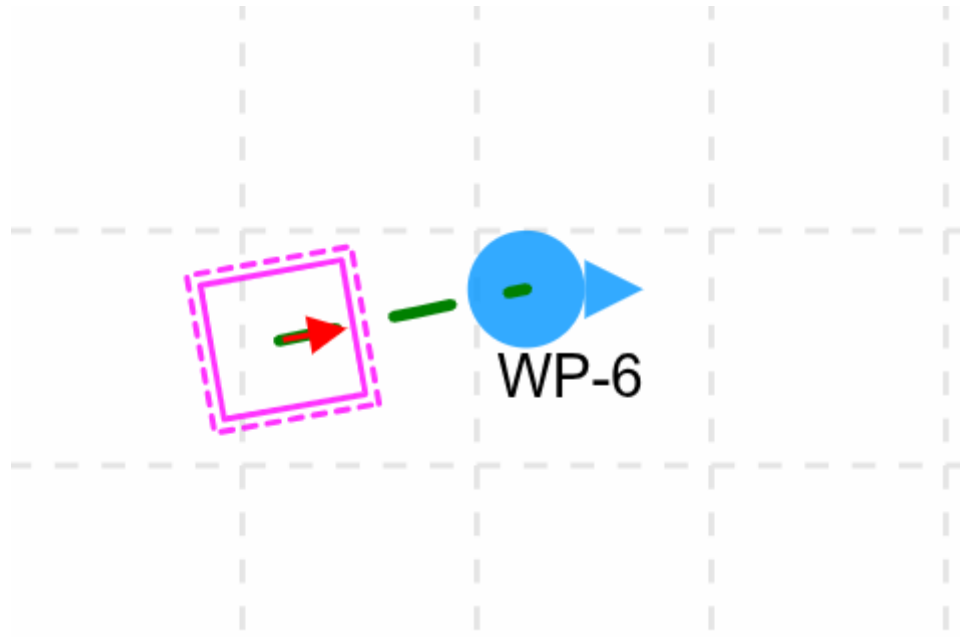
WP-7

Radius: 0.2

Then pressing the button *Play*, the robot receives the sequence of tasks to be carried out and starts the mission.

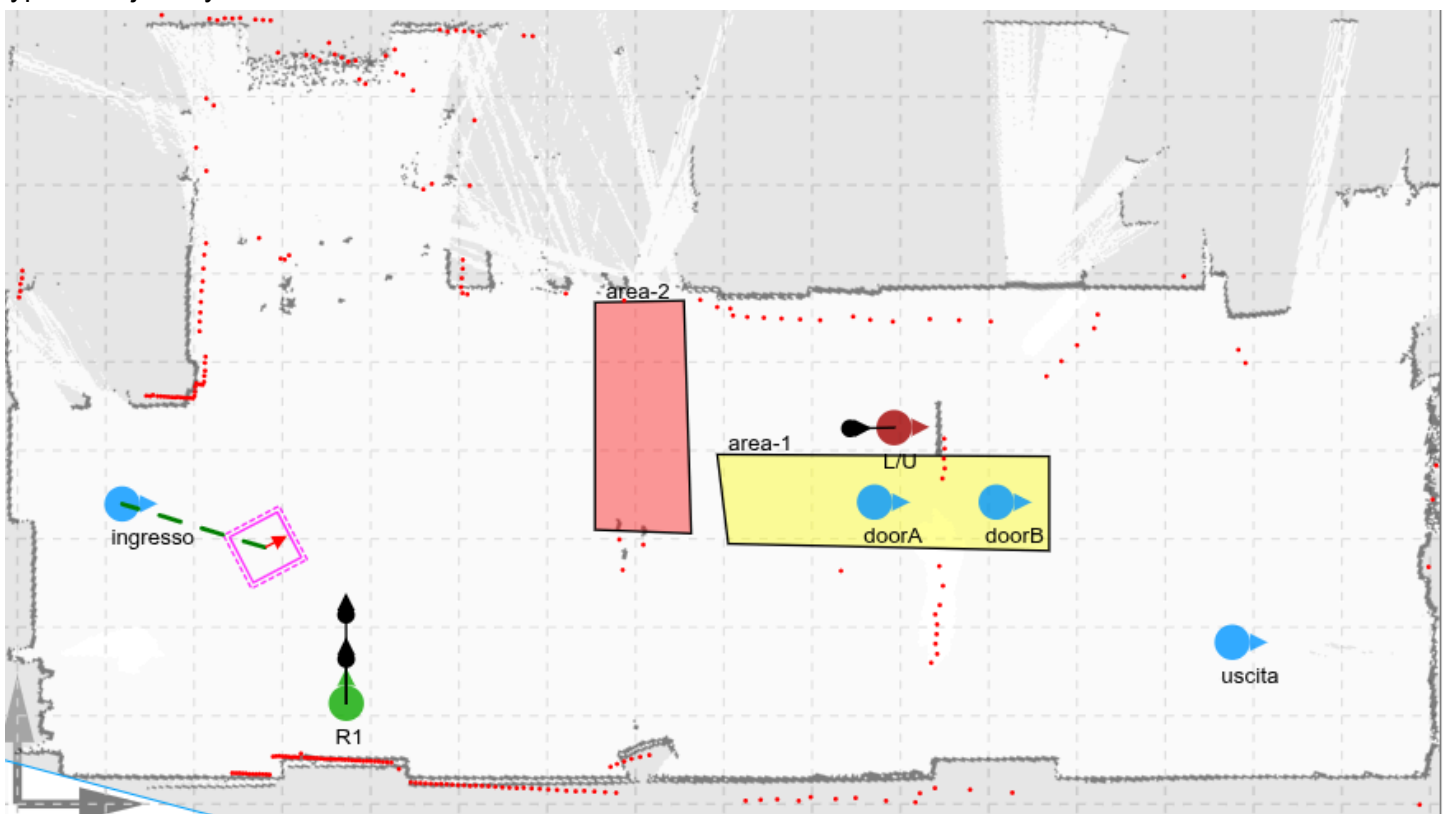


Once the mission is started, the robot's state will change from IDLE to RUN, and the user will be able to see the trajectory (dotted green in the map box) that the robot will follow during the mission.

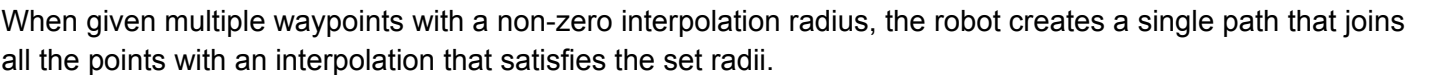
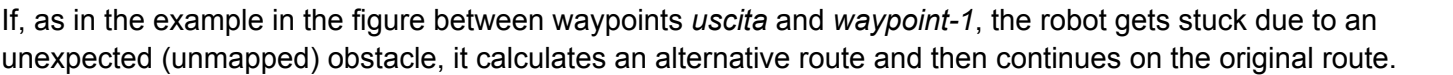


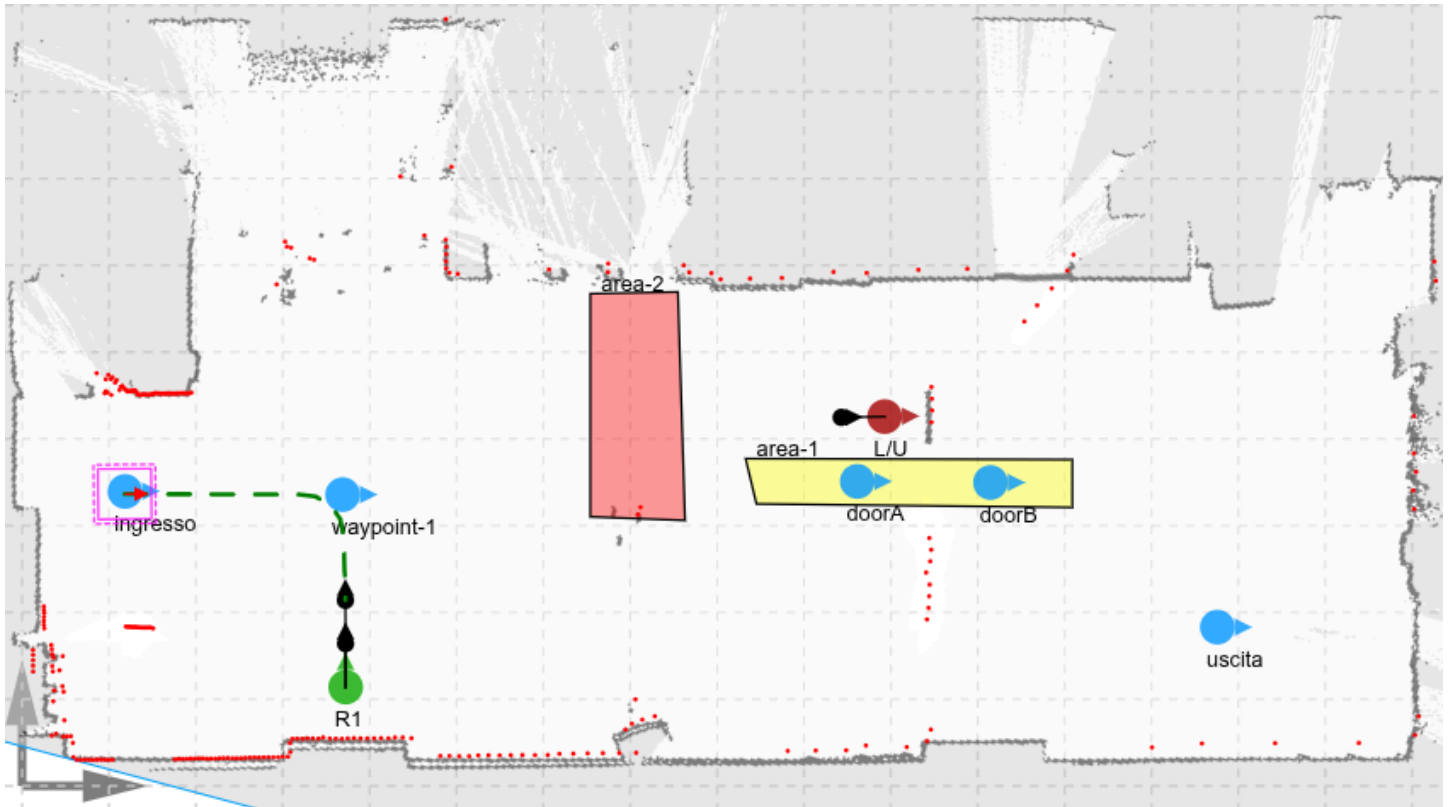
Possible Scenarios

Below are some examples of single- and multi-point missions, including map obstacles. When two points can be connected by a straight line with no static (mapped) obstacles between them, the robot will execute this type of trajectory.

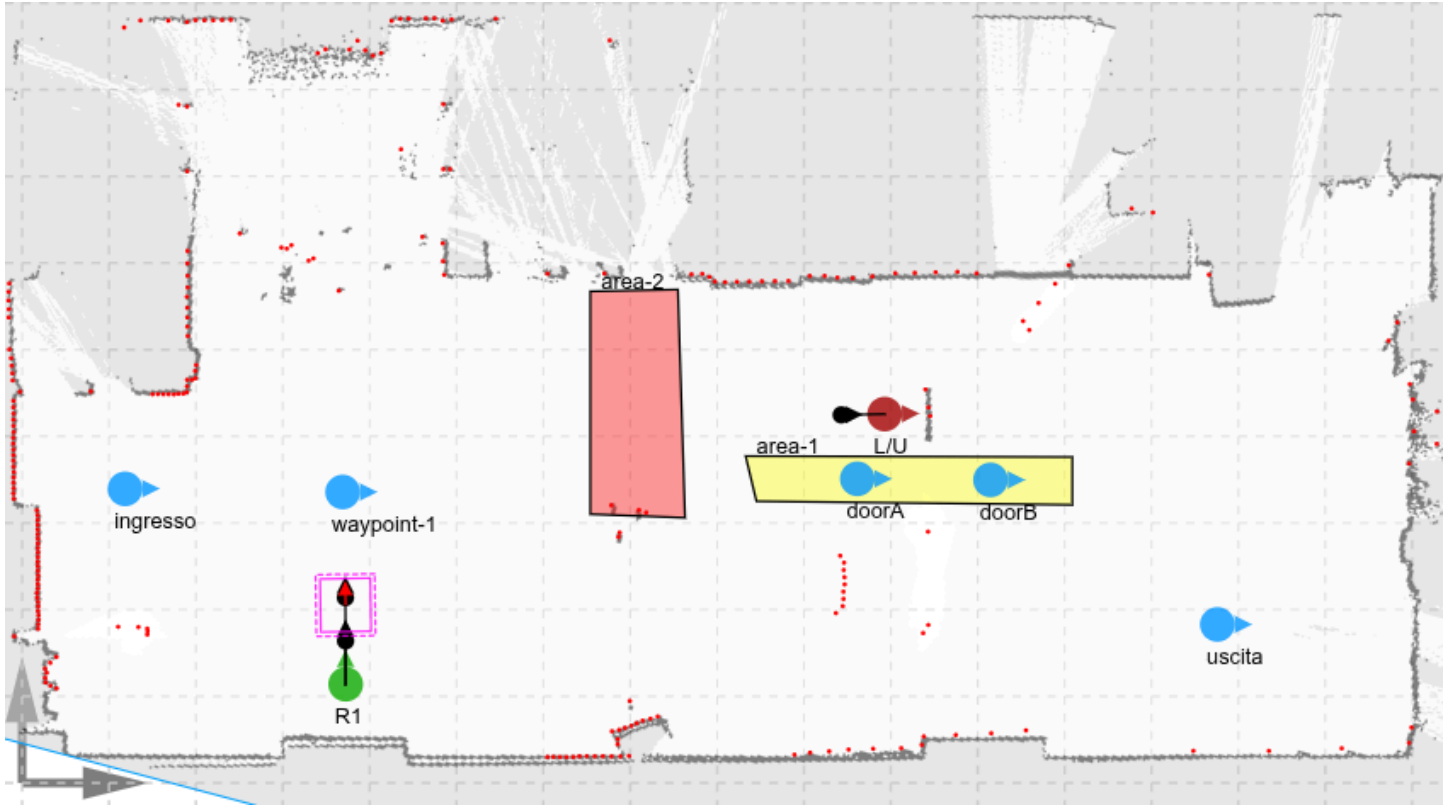


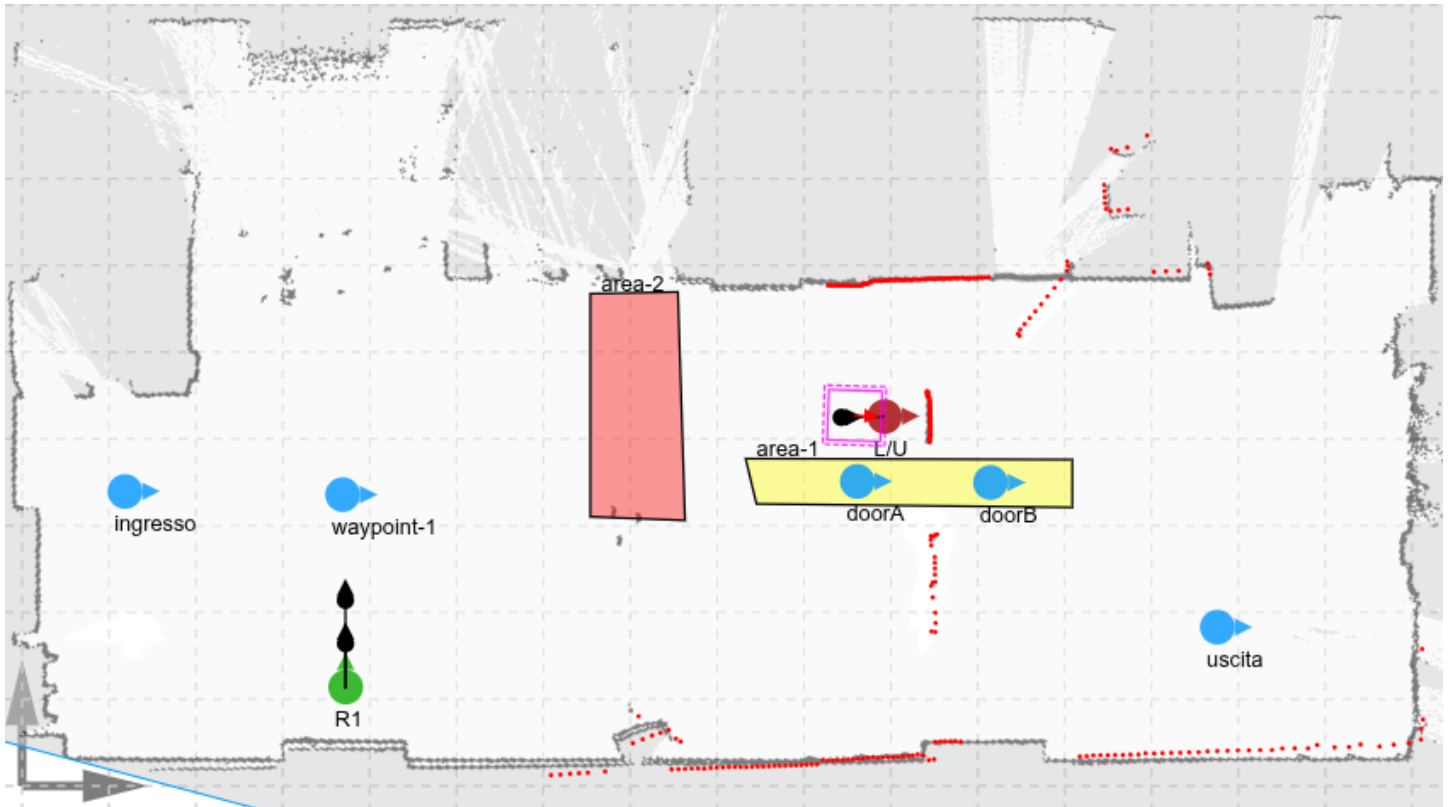
In the opposite case, when two points cannot be connected by a straight line with no static (mapped) obstacles between them, the robot will find an alternative route.



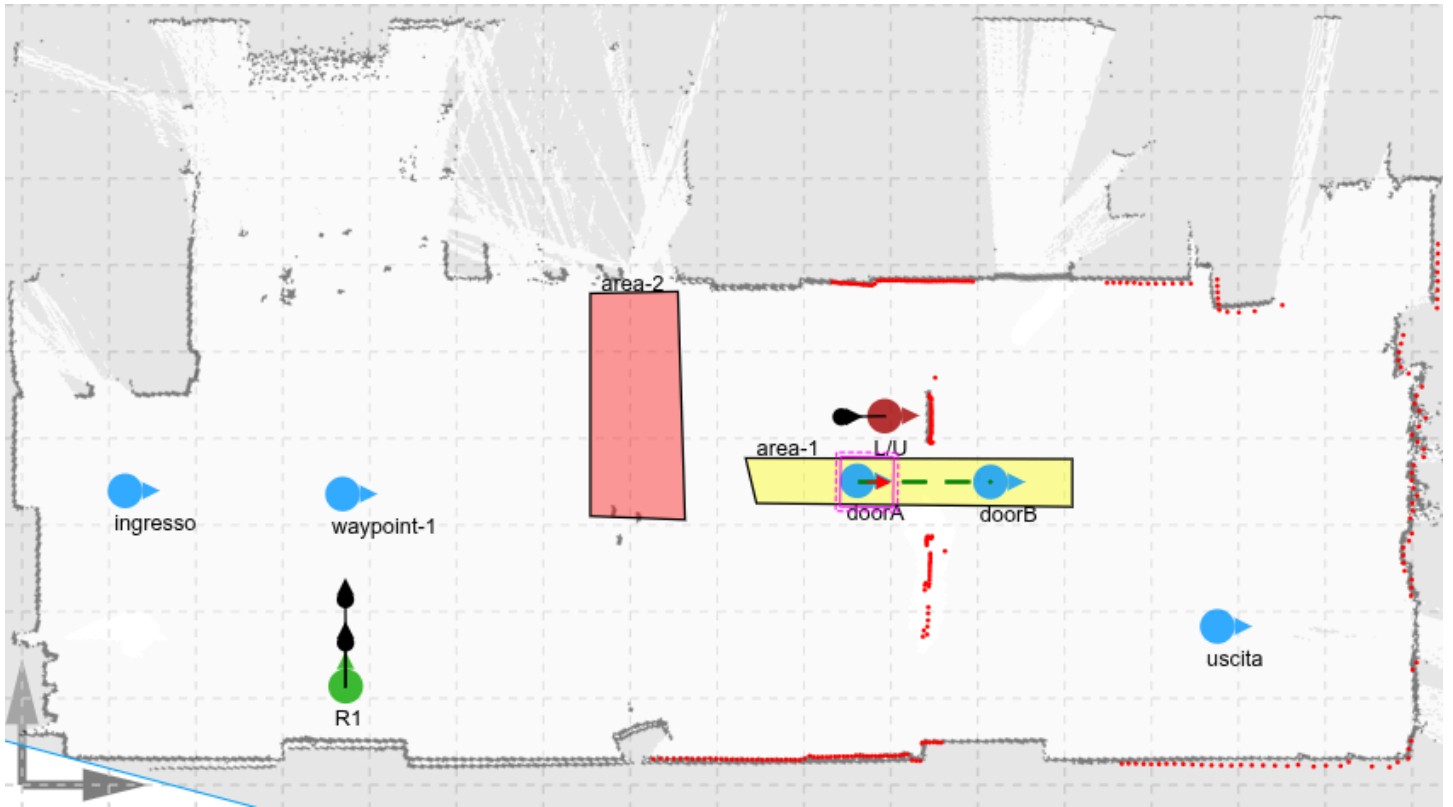


When a docking is requested either on a recharge or a load/unload point, the robot approach the point with a precision maneuver, depending on the type of docking selected.





When dealing with narrow passages, such as the one between the two obstacles in the yellow area, it's better to put a point before and after the passage, as done with doorA and doorB waypoints. In this way the robot won't try to find a route but will pass with a straight line, avoiding possible planning problems due to the precision of the lidar, the precision of the localization, and the precision of the path tracking.



Mission Control

Using the keys *Pause*, *Resume* and *Stop* you can control the flow of the mission.

The button *Pause* and the button *Resume* are mutually exclusive: it is not possible to pause a mission if the system is already in pause state, you can only resume the mission using the pause button *Resume* or abort the mission with the button *Stop*.



Conversely, the user will not be able to resume a mission that is already in progress, but only stop or pause it.



You can check the status of the robot through the “*Mission status*”, wording “*Robot status*”

Mission status			
Position	x (m)	y (m)	ψ (°)
	9.99	2.19	38.69
Velocity	x (m/s)	y (m/s)	ψ (rad/s)
Req	0	0	0
Actual	0	0	0.41
Status: PAUSE			
No errors			

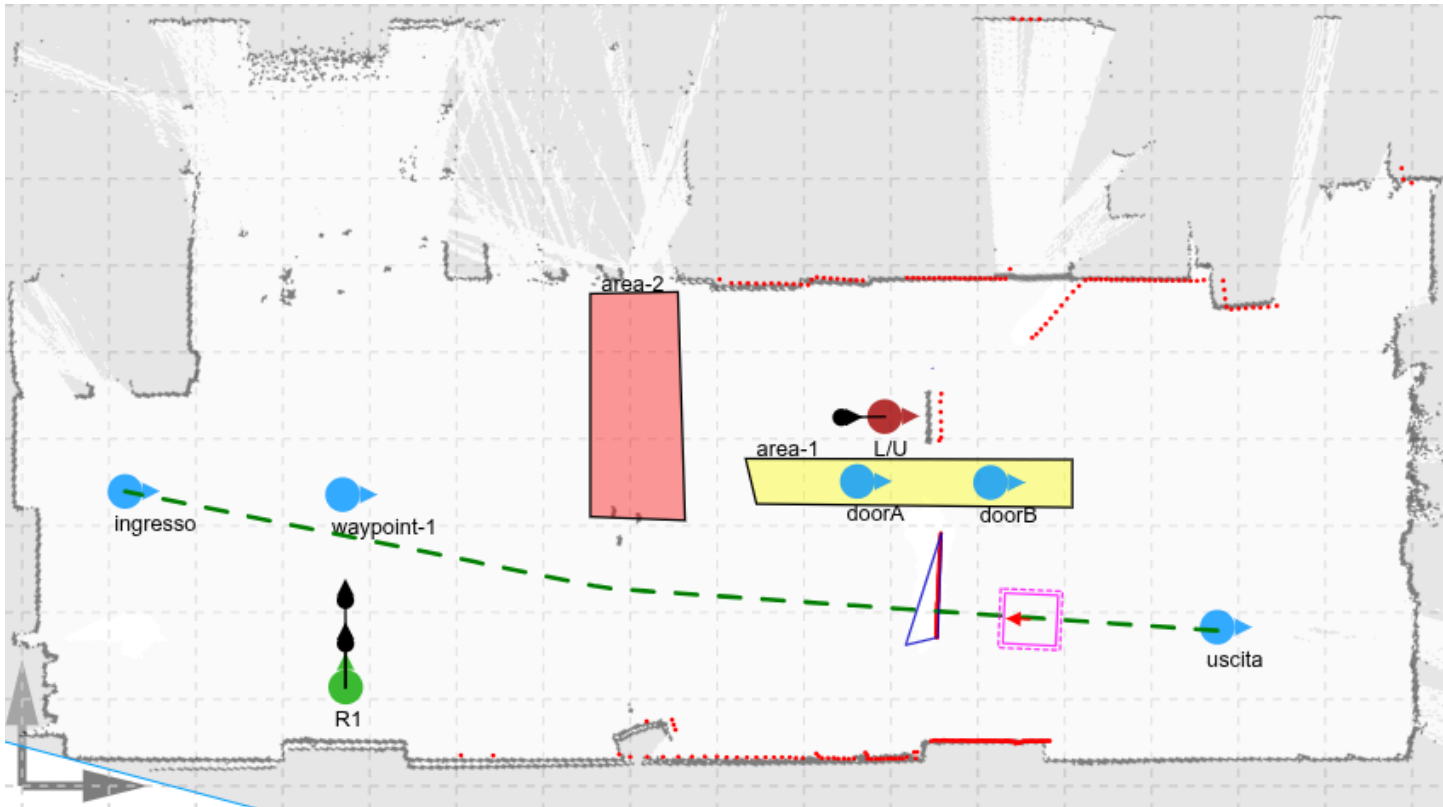
Properties

Show Obstacle

This property is to have feedback on unknown objects that are considered obstacle for the navigation system, they will appear as blue bounding boxes.

Properties

☐ Show Obstacles



Robot Status

In the section *Mission Status* the status appears (subsection *Robot Status*) of Proxima Compass Base, according to the following coding:

State	Meaning
IDLE	There are no missions in progress, the system is ready to receive a new mission
HEAT	The system is initializing; it is not yet possible to assign a mission
FINALIZE	The system is finalizing the mission
PAUSE	The system may be paused for 2 reasons: - the user pressed the button <i>Pause</i> and paused the running mission. To resume the mission you need to press the button <i>Resume</i>, otherwise to cancel the mission press the button <i>Stop</i>. - The system is reporting the SAFETY_ERROR error (see below for details), so the mission currently being executed is paused.
RUN	The system is performing the mission that was assigned by the user
ERROR	The system is in error; see below for details on error types.

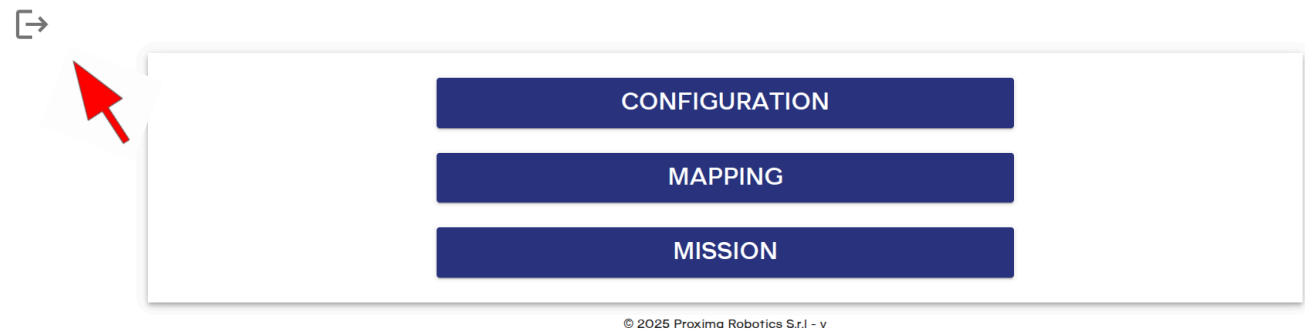
During the browsing experience, the system may report errors, which the user will be able to view on the screen in list format just in the section *Mission Status* subsection *Robot Errors*.

Mistake	Meaning
DRIVER_STUCK	The motor drivers have some anomalies that need to be checked.
LASER_ERROR	LIDAR data is not available.
LOCALIZATION_JUMP	A localization gap has been detected and a new localization is required.

LOCALIZATION_TIMEOUT	The location data is older than a certain time.
PATH NOT FOUND	No valid route to the point of interest was found. In this case, the Resume button will also be enabled to allow the user to try again to find an alternative route.
PLC_CONNECTION_ERROR	Connection errors to the PLC were detected.
ROBOT_OUT_OF_MAP	The robot has been located or moved off the map.
ROBOT_STUCK	The presence of obstacles around the robot prevents it from finding a valid alternative route.
SAFETY_ERROR	The safety control from the PLC (or equivalent instrumentation) prevents the robot from moving.

Correct logout procedure

From the home interface shown below



You can log out of the system by clicking the icon in the top left. You will then be returned to the home page.

ATTENTIONThis is the correct procedure to follow before shutting down the system. If you do not follow the correct procedure, you will be logged in for another minute. During this time, you will not be able to log in again.

ATTENTION: Currently, only one PC can be used at a time to connect to the interface.

Troubleshooting

Troubleshooting procedures for system-managed errors and possible anomalies are provided.

Mistake	Corrective action
LOCALIZATION_TIMEOUT	Locate the system using the specific procedure described in the section <i>Robot localization</i> .
SAFETY_ERROR	- If due to an operator stoppage (emergency button), wait for the operator to resolve the issue. - If due to an object/person in the safety sensors' monitoring area, remove the cause.
PATH NOT FOUND	- The robot is too large for the available path, check that the robot's footprint is correct - The point of interest is too close to obstacles, consider relocating the point of interest - The robot is positioned too close to obstacles at the start, manually

	move the robot to a clearer area • The robot is not localized correctly: check the localization in the graphical interface and, if necessary, run the system localization procedure.
--	---

Warnings and precautions

The Proxima Compass Base natural navigation system, although equipped with obstacle detection logic, is not a safety system. The user is responsible for ensuring their robot complies with current safety regulations.

Conclusion

This manual indicates the standard operations for using Proxima Compass Base. If you have any further needs, please contact customer support using the contact details provided.

Contacts

email: info@proximarobotics.com

web: <https://proximarobotics.com>

Appendix

Upload configuration

Below is an example syntax that is allowed for upload during setup:

```
{
  "general": {
    "robot_name": "agilex"
  },
  "connections": {
    "ip": "10.74.80.77",
    "cidr": 24
  },
  "kinematics": {
    "wheel_radius": 0.08,
    "reduction": 1,
    "axle_length": 0.49,
    "footprint_a": 0.306,
    "footprint_b": 0.306,
    "footprint_c_d": 0.29,
    "vel_lin_max": 0.5,
    "vel_ang_max": 0.5,
    "acc_lin_max": 0.3,
    "acc_ang_max": 0.35,
    "dec_lin_max": 0.3,
    "acc_lat_max": 0.1,
    "align_acc_the_max": 0.32,
    "align_vel_ang_max": 0.4,
    "align_well_right": 0.05,
    "padding": 0.05,
    "slowdown_area_vel_max": 0.3,
```

```
    "type": "diff_drive"
  },
  "obstacle": {
    "check_dynamic_obstacles": true,
    "obstacle_stop_distance": 0.5,
    "check_distance_out_of_path": true,
    "out_of_path_distance": 0.2,
    "stuck_recover_enabled": true,
    "stuck_recover_wait": 1,
    "out_of_path_recover_enabled": true,
    "out_of_path_recover_wait": 0
  },
  "motion": {
    "safety_topic": "/safety",
    "type": "ros2",
    "control_type": "twist",
    "diff_drive": {
      "odometry_status_topic": "/odom",
      "odometry_status_frame": "odom",
      "twist_cmd_topic": "/cmd_vel"
    }
  },
  "sensors": [
    {
      "x": 0.12,
      "and": 0,
      "With": 0.25,
      "yaw": 0,
      "type": "sick_nanoscan3",
      "ip": "10.74.80.7",
      "min_ang": -2.2602,
      "max_ang": 1.8762
    }
  ]
}
```